Energy-Efficient Orchestration in Heterogeneous Agentic Workflows via Small Language Models

Asadullah Nawaz Warraich
University of Helsinki
Helsinki, Finland
asadullah.warraich@helsinki.fi

***Abstract*—Agentic AI systems combine language models with tools and control loops to execute multi-step workflows, multiplying inference calls per user request. This paper proposes the *Green Agent Orchestrator* (GAO), a heterogeneous routing framework that decomposes agentic tasks into subtasks and assigns each to the smallest capable model from a pool of small language models (2B–9B parameters), while a homogeneous baseline uses a single 27B-parameter model for every step. We evaluated GAO on 15 deterministic benchmark tasks across three difficulty tiers (easy, medium, hard), with three runs per task, measuring energy consumption via CodeCarbon, task accuracy via deterministic evaluators, and wall-clock duration. The heterogeneous flow achieved a 75.0% reduction in energy consumption (0.135 mWh vs. 0.539 mWh per task) and a 73.4% reduction in execution time (33.4 s vs. 125.8 s), while matching or exceeding the baseline in accuracy (1.00 vs. 0.96). Energy savings ranged from 67.6% on medium tasks to 95.6% on easy tasks. These results demonstrate that heterogeneous orchestration with small language models can substantially reduce the energy footprint of agentic workflows without sacrificing task correctness.**

***Index Terms*—agentic workflows, small language models, energy efficiency, Green AI, orchestration, model routing**

I. Introduction

Language models can interleave reasoning with actions such as tool use, enabling multi-step trajectories rather than single-turn outputs [1], [2]. In practice, this pattern is implemented as a control loop that repeatedly calls a model to select tools, interpret results, and refine intermediate state [3]. Such *agentic workflows* can multiply inference calls per user request and amplify total compute.

The energy demand of AI-related compute has become a significant constraint. The International Energy Agency reports that electricity consumption from data centres and AI could more than double by 2026, reaching over 1 000 TWh [4], [5]. While early Green AI work focused on training cost [6], recent industry data show that inference now dominates the operational footprint of deployed models, accounting for roughly 60–70% of AI compute in large-scale services [7], [8]. Training is a one-time capital cost amortised over the model's lifetime, whereas inference recurs with every user interaction, so this paper concentrates on improving the energy efficiency of inference for already-deployed open-weight models [9].

Most agent architectures default to a *homogeneous* approach in which the same large model handles every step, including planning, tool calling, and final synthesis. This is potentially wasteful because many sub-tasks are structurally simple and may not require a large parameter budget. At the same time, naive replacement with a small model can reduce reliability when errors propagate across steps.

Prior work on multi-LLM *routing* and *cascading* tackles a related problem at the per-query level, sending each user query to one model chosen from a pool [10]–[12]. Recent work such as R2-Reasoner extends this idea by decomposing a reasoning task into subtasks and assigning each to a model of appropriate size [13]. Our work differs in two ways. First, we target *tool-augmented agentic workflows* rather than pure-text reasoning, so each sub-task may include external tool calls and the cost is measured in physical energy rather than API dollars. Second, we use lightweight prompted routing with no fine-tuning, making the approach immediately deployable on open-weight local models.

This paper addresses two research questions:

- **RQ1:** To what extent can heterogeneous routing reduce energy consumption in agentic workflows compared to a homogeneous single-model baseline?
- **RQ2:** How does routing sub-tasks to small language models affect end-to-end task accuracy?

We propose the *Green Agent Orchestrator* (GAO), a lightweight framework that decomposes tasks via a small orchestrator model (4B parameters), routes sub-tasks to the smallest capable worker model from a pool (2B, 4B, 9B), and compares against a homogeneous baseline using a single 27B-parameter model. We follow a constructive (artifact-building) research approach [14] and evaluate using CodeCarbon energy profiling [15], deterministic accuracy evaluators, and wall-clock timing across 15 benchmark tasks.

The results show that GAO reduces energy consumption by 75.0% and execution time by 73.4% without degrading task accuracy compared to the baseline. These findings demonstrate that heterogeneous orchestration with small language models is a practical strategy for energy-aware agent design.

Paper structure. Section II summarises background and related work. Section III describes the GAO design. Section IV specifies the evaluation method. Section V presents results and discussion. Section VI concludes.

II. Background and Related Work

A. Agentic Workflows and Tool Use

ReAct introduces an interleaving of reasoning traces and actions, showing that tool interaction can reduce hallucina-

tion and improve task completion in multi-step settings [1]. Toolformer demonstrates that language models can learn to decide when to invoke APIs and how to integrate results [2]. Gorilla further supports the idea that structured tool calls can be handled by models smaller than frontier-scale systems [16].

Surveys of LLM-based autonomous agents consolidate common agent components (planner, memory, tools, and control loop) and highlight evaluation challenges [17]. For this study, the key observation is that agentic execution is iterative, allocating substantial model capacity to control decisions that vary widely in difficulty.

B. Green AI and Measuring Energy

Green AI argues for reporting efficiency metrics as part of standard evaluation [9]. Earlier work quantifies the costs of large-scale deep learning and calls for more sustainable practices [6], [18]. A systematic review highlights increasing attention to monitoring and trade-off analysis in AI energy research [19]. Recent work warns that efficiency gains may be offset by increased usage through the Jevons paradox [20], [21].

CodeCarbon is a widely used open-source library that estimates energy use and CO_2 emissions from CPU, GPU, and RAM utilisation [15], [22]. Validation studies show that such estimators can have non-trivial error depending on hardware, implying that relative comparisons and careful reporting are important [23].

C. Routing and Cascades Across Models

Multi-LLM systems increasingly use *routing* (a single up-front choice of model) and *cascading* (sequential querying with escalation on low confidence) to match query complexity to model capability [12]. FrugalGPT formalised the cascade pattern and reported large cost reductions while maintaining quality [10]. Consistency-based cascades use answer agreement under sampling as a routing signal for reasoning tasks [11], and chain routing studies the balance between operational effectiveness and environmental cost in multi-model pipelines [14].

D. Sub-Task-Level Routing

A smaller body of work routes at the sub-task level rather than the query level. R2-Reasoner combines a Task Decomposer with a Subtask Allocator and trains both with reinforcement learning, reporting 84% API-cost savings on multi-step reasoning while preserving accuracy [13]. A recent survey notes that sub-task routing remains comparatively underexplored, particularly for tool-augmented agents [12]. The present paper sits in this space, but with three distinguishing choices: routing is performed by a small prompted orchestrator with no fine-tuning, the framework is evaluated on *agentic* workflows that include deterministic tool calls, and the primary cost metric is on-device energy rather than monetary API cost.

III. Green Agent Orchestrator Design

A. Design Goals

GAO targets a practical, reproducible implementation with four constraints: (i) runnable on a consumer laptop, (ii) compatible with open-weight quantised models served via Ollama, (iii) measurable with lightweight energy tooling (CodeCarbon), and (iv) evaluated with tasks that have objective success criteria to avoid expensive LLM-as-judge evaluation.

B. Architecture Overview

GAO is implemented as a LangGraph StateGraph with three stages. Figure 1 contrasts the heterogeneous flow with the homogeneous baseline.

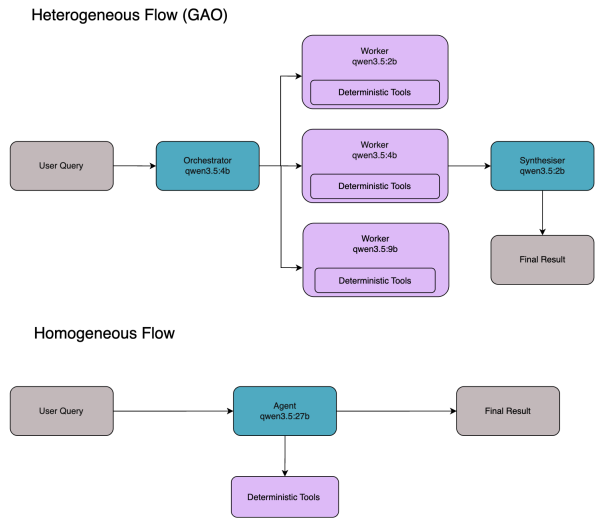

Fig. 1. Architecture of GAO. The orchestrator decomposes the query and routes sub-tasks to appropriately sized worker models. The baseline uses a single 27B model for all steps.

The **Orchestrator** (4B model) receives the user query and decomposes it into 1–4 self-contained sub-tasks using structured output via function-calling mode. Each sub-task is annotated with a difficulty level and assigned to the smallest sufficient worker model.

The **Workers** execute sub-tasks as ReAct agents [1] with access to five deterministic tools: calculator, unit converter, data lookup, date calculator, and text processor. Prior sub-task results are injected as context for sequential sub-tasks, ensuring information flows across the decomposed pipeline.

The **Synthesiser** (2B model) combines multi-subtask results into a coherent final answer. For single-subtask plans, synthesis is skipped entirely to eliminate an unnecessary LLM call.

The **Homogeneous baseline** uses a single 27B-parameter model as a ReAct agent with access to the same five tools. "Homogeneous" here means that the *same* model handles every step, not that the agent issues only one LLM call. The baseline still iterates through reason–act–observe cycles,

invoking the model once per planning step and once per tool-result assimilation, so simple tasks take 2 calls and complex tasks can take 5–8 (Section V).

C. Model Pool and Routing Policy

Table I summarises the model pool. The orchestrator assigns models based on estimated sub-task complexity. Simple lookups and single-tool tasks route to 2B, moderate multi-tool tasks to 4B, and complex reasoning tasks to 9B. A post-plan validation heuristic auto-upgrades assignments when the sub-task description contains multiple complexity indicators (e.g., three or more computation-related keywords), preventing small models from being assigned to tasks that exceed their capacity.

TABLE I
MODEL POOL CONFIGURATION. ALL MODELS USE THE QWEN3.5 ARCHITECTURE WITH 4-BIT QUANTISATION SERVED LOCALLY VIA OLLAMA.

Model	Role	Params	Flow
qwen3.5:27b-q4	Baseline agent	27B	Homo.
qwen3.5:4b	Orchestrator	4B	Hetero.
qwen3.5:2b	Worker / Synth.	2B	Hetero.
qwen3.5:4b	Worker	4B	Hetero.
qwen3.5:9b	Worker	9B	Hetero.

D. Energy-Saving Mechanisms and Implementation

The previous subsections describe the abstract design (decompose, route, execute, synthesise). The implementation realises this design with open-weight models served locally via Ollama, agent control flow built on LangChain and LangGraph [3], and energy attribution through CodeCarbon [15]. Tool implementations and library versions are listed in Appendix A.

Three implementation choices target energy reduction. First, the orchestrator (4B) and most workers (2B–4B) are substantially smaller than the baseline (27B), reducing per-call inference cost. Second, synthesis is skipped for single-subtask plans, eliminating one LLM call. Third, per-subtask step limits (10 for easy, 15 for medium, 25 for hard) prevent small models from entering wasteful reasoning loops on intractable subtasks. The thinking/reasoning mode of the Qwen3.5 model family was disabled (`reasoning=False`) for all models to ensure fair comparison and reduce token generation overhead.

IV. EVALUATION METHOD

A. Research Approach

We follow a constructive research approach in which an artifact (GAO) is built and evaluated against a baseline using quantitative measures. The evaluation compares a homogeneous flow (single 27B model) against a heterogeneous flow (GAO with 2B–9B models) on the same benchmark, measuring energy, accuracy, and time.

B. Benchmark Tasks

We designed 15 deterministic benchmark tasks across three difficulty tiers (Table II). Each task has predefined expected values and required tool calls, enabling automated evaluation without human judgement. Tasks span financial analysis, unit conversion, date arithmetic, text processing, and multi-step calculations. Easy tasks require a single tool invocation, medium tasks combine 2–3 tools, and hard tasks involve 4 or more tools with multi-step reasoning chains.

TABLE II
BENCHMARK TASK DISTRIBUTION WITH EXAMPLE TASKS PER TIER.

Tier	Description	Count
Easy	Single tool, single step (e.g., percentage calculation, unit conversion)	5
Medium	2–3 tools, moderate reasoning (e.g., multi-currency comparison, date offsets)	5
Hard	4+ tools, multi-step analysis (e.g., investment portfolio, loan amortisation)	5

C. Metrics

Three primary metrics are used:

- **Energy consumption** (kWh): measured by CodeCarbon [15] in offline mode, tracking CPU, GPU, and RAM power draw per task execution.
- **Accuracy**: a deterministic evaluator checks whether all expected values appear in the agent's response, with numeric tolerance for rounding.
- **Duration**: wall-clock time in seconds from task start to completion.

We also report CO_2 emissions (estimated by CodeCarbon using the Finnish electricity grid carbon intensity), number of LLM invocations, and *Energy-to-Solution* (EtS = energy / accuracy), which penalises incorrect results.

D. Experimental Setup

All experiments ran on an Apple MacBook Pro with M4 chip, 24 GB unified memory, and macOS 26.3. Models were served locally via Ollama with no network calls. Each task was executed three times per flow (90 total runs). For every task, both flows were run back-to-back before moving on to the next task, so that thermal state and Ollama model-caching effects affect both flows comparably. Temperature was set to 0.0 for deterministic inference, and the thinking mode of all Qwen3.5 models was disabled.

V. RESULTS AND DISCUSSION

A. Overall Comparison (RQ1)

Table III reports the main results broken down by difficulty tier (15 runs per cell), and the bottom section gives the pooled aggregate. Reporting per-tier means avoids the inflated standard deviations that would arise from averaging across difficulty tiers (e.g., easy and hard tasks differ by an order of magnitude in energy and duration), and ensures that the units in the table are consistent with Figures 2 and 4. Within each

tier the relative spread is moderate ($\sigma/\mu < 0.5$ for nearly all cells), so the per-tier means are a faithful summary.

TABLE III
PER-DIFFICULTY RESULTS (MEAN $\pm$ STD, $n = 15$ PER CELL). ENERGY IS REPORTED IN MWH TO MATCH FIG. 2.

Metric	Tier	Homo. (27B)	Hetero. (GAO)
Energy (mWh)	Easy	0.211 ± 0.078	0.009 ± 0.001
	Medium	0.337 ± 0.130	0.109 ± 0.059
	Hard	1.069 ± 0.367	0.285 ± 0.069
Duration (s)	Easy	60.6 ± 12.8	13.7 ± 0.9
	Medium	89.7 ± 17.7	25.9 ± 6.1
	Hard	227.0 ± 68.5	60.7 ± 9.4
LLM calls	Easy	2.0 ± 0.0	3.0 ± 0.0
	Medium	2.8 ± 0.8	7.4 ± 1.1
	Hard	5.3 ± 2.5	9.2 ± 2.0
Accuracy	Easy	1.00 ± 0.00	1.00 ± 0.00
	Medium	0.93 ± 0.14	1.00 ± 0.00
	Hard	0.93 ± 0.26	1.00 ± 0.00
All tiers	**Energy saving**	**75.0%**	
	Time saving	**73.4%**	

The pooled means (0.539 vs. 0.135 mWh and 125.8 vs. 33.4 s) confirm overall savings of 75.0% in energy and 73.4% in execution time across all 45 runs per flow. Total CO_2 emissions across all runs were 0.479 mg for the heterogeneous flow compared to 1.921 mg for the baseline.

The LLM-call counts in Table III also clarify a common misreading. The homogeneous baseline averages 2–5 calls per task (not one) because it executes a multi-step ReAct loop, and "homogeneous" refers only to using the *same* model at every step. The heterogeneous flow uses about 1.5–3$\times$ more calls (orchestrator plus per-subtask workers), yet still consumes far less energy because each call runs on a 2–4B model rather than a 27B model. This indicates that *model size per call* dominates *number of calls* in total energy.

B. Energy and Time Savings by Difficulty

Figure 2 shows absolute energy consumption by difficulty tier, and Figure 3 shows the corresponding percentage savings for both energy and time.

Easy tasks showed the largest energy savings (95.6%), where a single 2B worker replaced the 27B baseline for one-tool operations. Medium tasks achieved 67.6% savings, and hard tasks 73.3%. Time savings followed a similar pattern: 77.4% for easy, 71.1% for medium, and 73.3% for hard tasks (Figure 4).

The savings on hard tasks deserve emphasis. These tasks require 4–6 tool invocations and multi-step reasoning, yet the heterogeneous flow still achieved 73.3% energy and time savings. The orchestrator decomposed hard tasks into sub-tasks, routing data-fetching steps to 2B models and computation steps to 4B or 9B models. Even with the orchestrator overhead (one additional 4B inference call), the aggregate cost of multiple small-model calls was substantially lower than a single large-model execution.

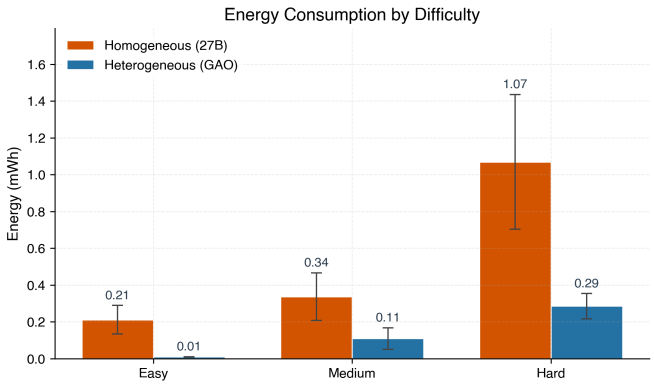

Fig. 2. Energy consumption by difficulty tier. The heterogeneous flow uses substantially less energy across all difficulty levels.

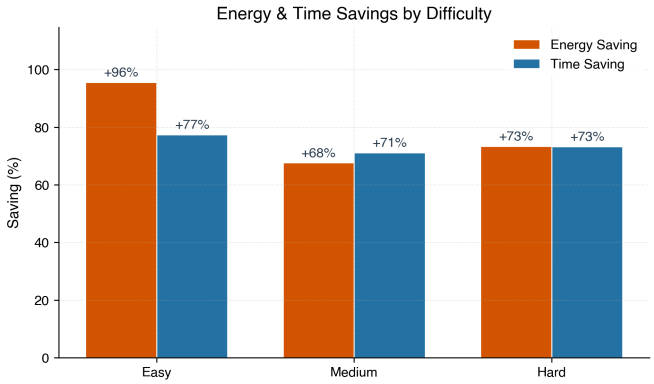

Fig. 3. Energy and time savings of the heterogeneous flow relative to the baseline, by difficulty tier. Savings exceed 67% across all tiers and both metrics.

Fig. 4. Execution time by difficulty tier. Hard tasks show the largest absolute reduction: from 227 s to 61 s.

C. Accuracy Analysis (RQ2)

The heterogeneous flow achieved perfect accuracy (1.00) across all 45 runs, while the homogeneous baseline achieved 0.96 (Figure 5). The baseline's lower accuracy stemmed from occasional failures on medium and hard tasks (0.93 each), where the 27B model sometimes produced incorrect tool arguments or omitted required computation steps.

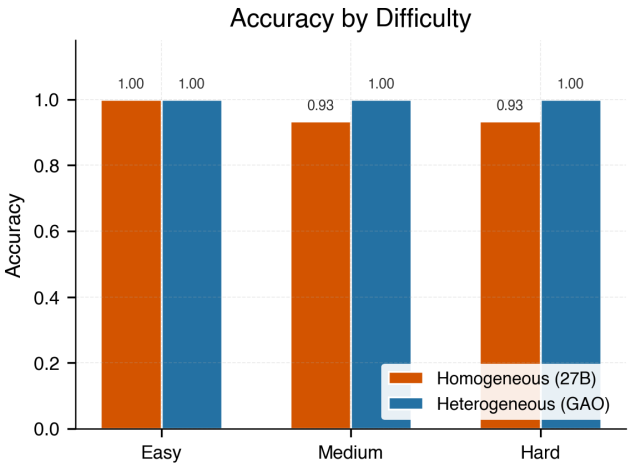

Fig. 5. Accuracy by difficulty tier. The heterogeneous flow achieves perfect accuracy across all tiers, while the baseline drops to 0.93 on medium and hard tasks.

This result addresses RQ2. Routing to small models did not degrade accuracy. The orchestrator's task decomposition breaks complex queries into simpler, self-contained sub-tasks that small models can handle correctly, so each individual inference step becomes easier and compensates for the smaller model capacity. We assess the statistical significance of the small accuracy improvement (4 baseline failures out of 45 runs) in Section V-E. The Energy-to-Solution metric (energy divided by accuracy) reflects both the large energy reduction and the unchanged-or-better correctness, falling from 0.564 mWh for the baseline to 0.135 mWh for the heterogeneous flow.

D. Model Usage Distribution

Figure 6 shows the LLM call distribution within the heterogeneous flow, computed across all 294 model invocations in the 45 evaluation runs. The 2B model accounted for 52.0% of calls (153 invocations as worker or synthesiser), the 4B model for 42.9% (126 invocations including the orchestrator on every run plus 4B worker assignments), and the 9B model for only 5.1% (15 invocations). Approximately 95% of all calls used models with 4B parameters or fewer.

This distribution validates the core design hypothesis that most agentic sub-tasks are structurally simple and can be handled by small models. The 9B model was only invoked 15 times, almost exclusively on hard tasks involving complex

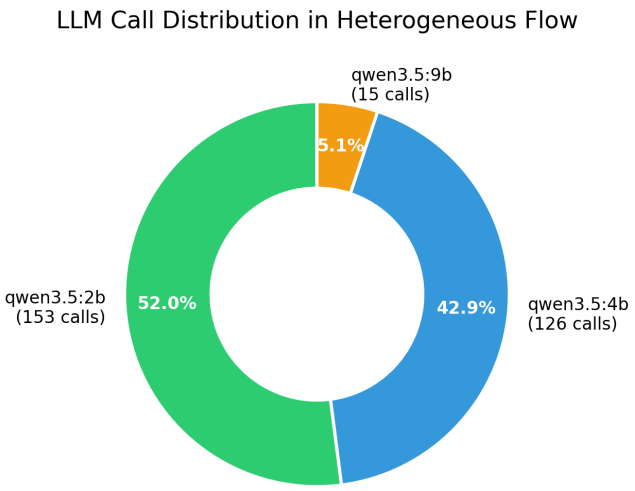

Fig. 6. LLM call distribution in the heterogeneous flow across all 294 invocations. About 95% of calls use 4B-parameter models or smaller; the 9B model is reserved for the few sub-tasks that exceed the operating range of the 4B worker.

multi-step financial calculations. The remaining 279 invocations were handled by the 2B and 4B models, which are 3–14× smaller than the baseline model.

E. Discussion

The results provide strong evidence for both research questions. For RQ1, the heterogeneous flow achieved consistent energy savings across all difficulty tiers (67.6–95.6%), with an overall reduction of 75.0%. The savings are primarily driven by the difference in model size. The 27B baseline requires substantially more compute per inference call than the 2B–4B workers, and this per-call difference compounds over the 3–7 invocations typical of agentic workflows.

For RQ2, the decomposition strategy preserved accuracy: GAO produced a correct answer on every one of the 45 runs (1.00) while the baseline failed on 4 runs out of 45 (0.91 on medium and hard tasks combined). A two-sided Fisher's exact test on this 45/45 vs. 41/45 contingency yields $p \approx 0.12$, so the accuracy improvement is suggestive but not statistically significant at $\alpha = 0.05$ given the modest sample. We therefore frame the accuracy result conservatively, claiming that routing to small models did *not degrade* accuracy, rather than that it strictly improved it.

Routing errors and recovery. The orchestrator's routing decisions were not infallible. In early development we observed cases where a 2B model received a sub-task that involved a long arithmetic chain or multi-currency comparison and made order-of-operations errors. We mitigated this with a post-plan validation heuristic that auto-upgrades 2B assignments to 4B when the sub-task description contains three or more computation-related keywords or exceeds 160 characters (Section III-C). With this heuristic, no 2B routing failure occurred

in the 45 evaluation runs. We do not claim this fully solves the routing-error problem. On out-of-distribution tasks, escalation policies similar to those used in cascades [10], [11] would likely be needed.

Comparison to prior work. Our overall energy savings (75.0%) are in the same range as the cost savings reported by query-level cascades such as FrugalGPT (up to 98% on selected tasks) [10] and the API-cost savings of R2-Reasoner (84.5%) [13], but in our setting the cost is physical energy on a single machine rather than dollars across cloud APIs. Compared to query-level routing [10], [11], GAO routes at the *sub-task* level inside a single query, which is more appropriate for agentic workflows because different steps within one query have very different complexity profiles. Compared to R2-Reasoner [13], GAO uses a small prompted orchestrator with no fine-tuning, which makes it directly deployable on local open-weight models. The finding that roughly 95% of LLM calls in the heterogeneous flow are handled by 4B-or-smaller models confirms the central premise of routing surveys [12], namely that many sub-tasks in real workflows do not need a frontier-scale model.

Benchmark design. We chose to design 15 deterministic benchmark tasks with known expected answers, rather than reusing an existing reasoning benchmark, for two reasons. First, we needed objective accuracy evaluation without an LLM-as-judge, which is itself energy-intensive and would confound the energy measurement. Second, the focus of the study is the *agentic* pattern, in which tasks combine multiple tool calls with arithmetic, lookups, and synthesis. Existing reasoning benchmarks (e.g., GSM8K, MATH) target pure-text reasoning rather than tool-augmented multi-step trajectories. We acknowledge that hand-designing tasks risks bias, and the perfect heterogeneous accuracy may partly reflect that the task complexity stays within the operating range of a 4B-or-smaller model. Replicating the study with tasks drawn from existing tool-use benchmarks (e.g., ToolBench, AgentBench) is a clear next step.

Reproducibility. The full source code, benchmark task definitions, raw measurement traces, and analysis scripts are publicly available at https://github.com/Asadullah378/Green-Agent-Orchestrator under the MIT license.

F. Threats to Validity

Construct validity. CodeCarbon estimates energy from hardware counters and may differ from direct power measurements [23]. We mitigate this by reporting relative comparisons on identical hardware and averaging over three runs.

Internal validity. The execution order alternated flows per task to control for thermal state and Ollama model caching. Deterministic tools and temperature = 0 reduce run-to-run variance. Residual variance stems from OS scheduling and background processes.

External validity. The benchmark uses five deterministic tools with hardcoded data, whereas real-world tasks involve noisier tools and open-ended queries. The model pool is limited to the Qwen3.5 family, and results may differ with other architectures or quantisation levels. All experiments ran on Apple Silicon, and energy profiles may differ on x86 or discrete-GPU hardware.

VI. Conclusion

This paper proposed the Green Agent Orchestrator (GAO), a heterogeneous routing framework that assigns agentic workflow sub-tasks to the smallest capable model from a pool of small language models. Evaluated on 15 deterministic benchmark tasks with three runs each, GAO reduced energy consumption by 75.0% and execution time by 73.4% compared to a homogeneous 27B-parameter baseline, while preserving end-to-end accuracy (1.00 vs. 0.96; the difference is not statistically significant by Fisher's exact test, $p \approx 0.12$). Approximately 95% of heterogeneous LLM calls used models with 4B parameters or fewer, demonstrating that most agentic sub-tasks do not require large models.

These results support the Green AI principle that efficiency should be a first-class design criterion [9]. The orchestration overhead is modest because the 4B orchestrator adds only a single fast inference call, which is more than offset by the cumulative savings from routing workers to 2B–4B models.

Two limitations should be noted. First, the benchmark tasks are deterministic and bounded, while real-world agentic tasks involve open-ended reasoning where small models may be less reliable. Second, energy measurements rely on CodeCarbon estimates rather than physical power meters.

Future work could extend GAO to dynamic routing with confidence-based escalation [11], test on open-ended coding and web-browsing benchmarks, and validate energy measurements against hardware power meters [23]. Ensuring that efficiency gains translate to absolute reductions rather than increased usage, as cautioned by the Jevons paradox literature [20], is an important direction for sustainable AI research.

References

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," *arXiv preprint*, 2022. [Online]. Available: https://arxiv.org/abs/2210.03629

[2] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," *arXiv preprint*, 2023. [Online]. Available: https://arxiv.org/abs/2302.04761

[3] LangChain contributors, "Agents – langchain documentation," Documentation page, 2024, accessed: 2026-02-21. [Online]. Available: https://docs.langchain.com/oss/javascript/langchain/agents

[4] International Energy Agency, "Executive summary – electricity 2024," IEA report webpage, 2024, accessed: 2026-02-21. [Online]. Available: https://www.iea.org/reports/electricity-2024/executive-summary

[5] ——, "Electricity 2024: Analysis and forecast to 2026," IEA report (PDF), 2024, accessed: 2026-02-21. [Online]. Available: https://iea.blob.core.windows.net/assets/ddd078a8-422b-44a9-a668-52355f24133b/Electricity2024-Analysisandforecastto2026.pdf

[6] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in nlp," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019. [Online]. Available: https://aclanthology.org/P19-1355/

[7] D. Patterson, J. Gonzalez, U. Hölzle, Q. Le, C. Liang, L.-M. Munguia, D. Rothchild, D. So, M. Texier, and J. Dean, "The carbon footprint of machine learning training will plateau, then shrink," *Computer*, vol. 55, no. 7, pp. 18–28, 2022. [Online]. Available: https://doi.org/10.1109/MC.2022.3148714

[8] J. Fernandez, L. Wehrstedt, V. Shah, Y. Du, Y. Tsvetkov, and E. Strubell, "Hardware scaling trends and diminishing returns in large-scale distributed training," *arXiv preprint*, 2025. [Online]. Available: https://arxiv.org/abs/2504.17674

[9] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, "Green ai," *Communications of the ACM*, vol. 63, no. 12, pp. 54–63, 2020. [Online]. Available: https://dl.acm.org/doi/10.1145/3381831

[10] L. Chen, M. Zaharia, and J. Zou, "Frugalgpt: How to use large language models while reducing cost and improving performance," *arXiv preprint*, 2023. [Online]. Available: https://arxiv.org/abs/2305.05176

[11] M. Yue, J. Zhao, M. Zhang, L. Du, and Z. Yao, "Large language model cascades with mixture of thought representations for cost-efficient reasoning," in *International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/file/5de11e930c1bbfda5d4fc9d2b0924032-Paper-Conference.pdf

[12] C. Varangot-Reille, C. Bouvard, A. Gourru, M. Bouchard, P. Gallinari, and F. Jacquenet, "Doing more with less: A survey on routing strategies for resource optimisation in large language model-based systems," *arXiv preprint*, 2025. [Online]. Available: https://arxiv.org/abs/2502.00409

[13] B. Shao, Y. Liu, Y. Wu, T. Yu, W. X. Zhao, and J.-R. Wen, "R2-reasoner: Reinforced model router for divide-and-conquer multi-step reasoning," *arXiv preprint*, 2025. [Online]. Available: https://arxiv.org/abs/2506.05901

[14] S. Wheeler, M. Torgbi, and H. Tayyar Madabushi, "Reducing environmental costs whilst maintaining operational effectiveness in large language models through chain routing," in *KONVENS*, 2025. [Online]. Available: https://aclanthology.org/2025.konvens-2.20.pdf

[15] B. Courty, V. Schmidt *et al.*, "mlco2/codecarbon: v2.4.1," Zenodo software record, 2024. [Online]. Available: https://doi.org/10.5281/zenodo.11171501

[16] S. G. Patil, T. Zhang *et al.*, "Gorilla: Large language model connected with massive apis," *arXiv preprint*, 2023. [Online]. Available: https://arxiv.org/abs/2305.15334

[17] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, 2024. [Online]. Available: https://arxiv.org/abs/2308.11432

[18] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, "On the dangers of stochastic parrots: Can language models be too big?" in *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 2021, pp. 610–623. [Online]. Available: https://dl.acm.org/doi/10.1145/3442188.3445922

[19] R. Verdecchia, P. Malakar, P. Lago, V. Cortellessa, and G. Procaccianti, "A systematic review of green ai," *WIREs Data Mining and Knowledge Discovery*, 2023. [Online]. Available: https://robertoverdecchia.github.io/papers/WIRE_2023.pdf

[20] A. S. Luccioni, E. Strubell, and K. Crawford, "From efficiency gains to rebound effects: The problem of jevons' paradox in ai's polarized environmental debate," *arXiv preprint*, 2025. [Online]. Available: https://arxiv.org/abs/2501.16548

[21] S. Lange, F. Kern, J. Peuckert, and T. Santarius, "The jevons paradox unravelled: A multi-level typology of rebound effects and mechanisms," *Energy Research & Social Science*, vol. 74, p. 101982, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S221462962100075X

[22] CodeCarbon contributors, "Codecarbon: Track and reduce co2 emissions from your computing," Project website, 2024, accessed: 2026-02-21. [Online]. Available: https://codecarbon.io/

[23] R. Fischer, "Ground-truthing ai energy consumption: Validating codecarbon against external measurements," *arXiv preprint*, 2025. [Online]. Available: https://arxiv.org/abs/2509.22092

Appendix A
Implementation Details

Frameworks. GAO is built on `LangChain 0.3` and `LangGraph` for agent control flow [3], `langchain-ollama` for local model serving, `Pydantic` for structured-output schemas, and `CodeCarbon` [15] for per-task energy attribution. All models are quantised (4-bit, `q4_K_M`) and served by `Ollama 0.5` on the same machine, and no network calls leave the host.

Tools available to all agents. The five deterministic tools shared by both flows are (i) `calculator` (safe-AST evaluation of numeric expressions, supporting `+`, `-`, `*`, `/`, `**`, `^`); (ii) `unit_converter` (length, mass, temperature, currency with hard-coded rates); (iii) `data_lookup` (mock employee/product/customer records); (iv) `date_calculator` (date arithmetic with weekday awareness); and (v) `text_processor` (word count, case conversion, simple analysis). All tools are deterministic so that accuracy variance comes only from the model, not the tool.

Routing heuristic. The orchestrator emits structured JSON containing, for each sub-task, a free-text description, a difficulty label (*easy*/*medium*/*hard*), and an assigned model. A post-plan validator scans each description for computation-related keywords (e.g., *compute*, *calculate*, *compare*, *convert*, *aggregate*) and upgrades any 2B assignment to 4B if three or more such keywords appear or the description exceeds 160 characters.

Appendix B
Response to Reviewers

We thank the reviewers for the thoughtful and constructive feedback. The major changes in this revision are: (1) Table III now reports per-difficulty results with consistent mWh units (Review 3B-1,3B-3); (2) the introduction and related-work sections distinguish training and inference energy and add a sub-task-routing paragraph (Review 3B); (3) the discussion adds Fisher's exact test, a benchmark-design rationale, a routing-error analysis, and a reproducibility statement (Reviews 3B and 3C); (4) Figure 3 is recoloured to match the rest of the paper (Review 3B). We list per-comment responses below.

B.1 Review #3B

High std in Table III, units, "how can homogeneous have >1 LLM call". Addressed in the paper. Table III now splits results by difficulty tier, which removes the cross-tier variance that inflated the previously reported standard deviations (the within-tier coefficient of variation is below 0.5 in nearly all cells). All energy values in the table are in mWh and now match Fig. 2 numerically. Section III-B and the paragraph following Table III explain that "homogeneous" refers to using the *same* model at every step of the ReAct loop rather than to issuing a single LLM call. The baseline averages 2–5 calls per task because each plan/observe step is one model invocation.

Training vs. inference energy. Added in the introduction and related work, with citations to recent work showing that inference accounts for roughly 60–70% of operational AI energy [7], [8]. We note that training is a one-time cost that is amortised over the deployment lifetime, whereas inference recurs with every interaction. This paper deliberately targets the inference side of the equation.

Intro paragraph on related work / depth on sub-task routing. Added a differentiation paragraph at the end of the introduction and a dedicated subsection (II-D, *Sub-Task-Level Routing*) that discusses R2-Reasoner [13] and the routing survey of Varangot-Reille et al. [12]. The discussion section now explicitly relates our 75% energy savings to FrugalGPT's cost savings and R2-Reasoner's 84.5% API-cost savings.

Why custom benchmark tasks; were they too easy? Discussed in Section V-E under *Benchmark design*. We chose deterministic tasks to avoid LLM-as-judge evaluation (which would itself consume substantial energy and confound the measurement) and because existing reasoning benchmarks such as GSM8K and MATH target pure-text reasoning rather than tool-augmented agentic trajectories. We acknowledge in the same paragraph and in the threats-to-validity that the perfect heterogeneous accuracy is consistent with the tasks staying within the operating range of a 4B-or-smaller model, and we list replication on ToolBench/AgentBench as an explicit next step.

Different colours in Fig. 3. The figure now uses the same orange/blue palette as the rest of the paper (orange for energy saving and blue for time saving), so the colour scheme is consistent across figures. The y-axis is in percent rather than absolute mWh because both bars represent relative improvements of GAO over the baseline, and the caption states this explicitly.

B.2 Review #3A

Heuristics for task complexity (III-C). The complexity heuristic is the number of tool invocations required: easy = one tool, medium = 2–3 tools, hard = 4 or more and was already mentioned in Table II. This is now stated explicitly in Section IV-B. The orchestrator uses the same notion at runtime via its difficulty label, and the post-plan validator (Section III-C and Appendix A) catches under-routing of complex sub-tasks to 2B.

Introductory paragraph before tools/frameworks; synthesis of the section. Section III-D now opens with a short paragraph that distinguishes the abstract design (decompose, route, execute, synthesise) from the concrete implementation (LangGraph, Ollama, CodeCarbon), and forwards detailed tool descriptions to Appendix A.

Distribution of the data (RQ1). The per-tier breakdown in Table III and the per-tier means with error bars in Figs. 2–4 provide clearer descriptive statistics. A formal distributional analysis (e.g., fitting parametric distributions to the energy traces, or hierarchical modelling) would be a deeper study; we treat it as out of scope for this short paper and a candidate for follow-up work.

Deterministic-only tasks (RQ2). We also flag this limitation in Section V-E and the threats to validity. Adding non-deterministic tasks would require an alternative evaluator (e.g., LLM-as-judge), which would itself consume energy and is a separate methodological problem; we therefore defer it to future work as discussed in the conclusion.

Reasoning mode and architectural asymmetry. The reviewer correctly notes that the orchestration step itself is a form of structured reasoning that the homogeneous baseline does not have. We acknowledge that this is an inherent property of the heterogeneous architecture rather than something to be eliminated, because the contribution of the paper is precisely the claim that this lightweight structured decomposition is cheaper than running a large model at every step. The accuracy difference is small and, as reported in Section V-E, not statistically significant by Fisher's exact test, so we do not claim that orchestration "adds" reasoning capability beyond what fair routing requires.

B.3 Review #3C

Tasks designed by the author / routing strategy designed by the author. Discussed in Section V-E. We agree with the concern, retain it in the threats to validity, and propose replication on existing tool-use benchmarks as a direct next step.

Statistical testing of the accuracy difference. Added in Section V-E. A two-sided Fisher's exact test on 45/45 vs. 41/45 yields $p \approx 0.12$. We have weakened the wording from "equal or higher" to "did not degrade" accuracy.

Routing errors. Discussed in Section V-E under *Routing errors and recovery*. During development we observed 2B order-of-operations failures on long arithmetic chains, and the post-plan validator (Appendix A) auto-upgrades such sub-tasks to 4B and prevented all such failures in the 45 evaluation runs.

Reproducibility. Added a reproducibility statement at the end of Section V-E pointing to the public GitHub repository containing source code, benchmark definitions, raw measurement JSON files, and analysis scripts under the MIT license.

Single model family / generalisation across architectures. Acknowledged in the threats to validity (external validity). Replicating with non-Qwen families (e.g., Llama-3 1B/8B/70B, Mistral) is listed as future work and is not within the page budget of this short paper.

Minor: "GAO is implemented" missing space, Qwen3.5 release info. The original sentence reads "GAO is implemented" (`\systemname{} is implemented`), which renders with a regular space, and we have re-checked the source. The model tags follow the strings used by Ollama (e.g., `qwen3.5:27b-q4_K_M`), which are 4-bit quantised checkpoints from the Qwen-family series served locally. We have clarified this in Table I's caption.